

Internal APIs Are All You Need

A Shared Route Graph for Agent Web Access:
Discover Once, Reuse Forever, Settle over x402

Lewis Tham
Unbrowse AI
lewis@unbrowse.ai

Cayden Chik
Unbrowse AI
cayden@unbrowse.ai

April 2026 (arXiv:2604.00694)

Abstract

Most AI agents use the web the way a tired human would: open the page, wait for it to load, fight a popup, click through a menu, wait again. Underneath that ceremony every site already speaks a structured request language — the internal endpoints its own front-end calls. This paper makes one claim and tries not to oversell it: for the overwhelming majority of agent web tasks, you do not need a browser per call; you need to learn the internal request path *once* and reuse it. We frame the web as a shared *route graph*: an intent resolves to a ranked endpoint shortlist, the chosen route is executed for real data, and the mapping cost is amortised across every later agent that asks the same question. The economic core is a single inequality — reuse wins whenever the fee for replaying a mapped route undercuts the cost of rediscovering it from a browser — and we settle paid execution over **x402** so the indexer who mapped a route, the platform, and the site owner are all paid in one on-chain transaction. We report a peer-reviewed benchmark across 94 live domains and label every claim **[shipped]**, **[reference]**, or **[proposed]** so the reader is never sold a roadmap as a result. The short version: internal APIs are a great first layer — they are most of what an agent needs to read and act on the web. What they are *not* is the whole stack, and the companion papers take up where this one stops.

1 The browser-discovery tax

The first problem an agent has on the open web is *discovery*: finding the route in the first place is expensive, and re-deriving it on every call is wasteful. A browser session loads megabytes of markup, executes the page’s JavaScript, and waits on late XHR settle — per agent, per call — to recover a request the site’s own front-end already knew how to make. This is a tax paid over and over for information that does not change between callers. The thesis of this paper is that the tax is avoidable: the structured request behind a page is *learnable once* and *replayable by anyone*, so the discovery cost should be paid a single time and amortised, not re-paid on every visit.

2 The shared route graph

We model the web an agent can act on as a graph of reusable routes. A route is a callable endpoint with its method, parameters, auth requirements, and a typed record of what it returned. The two-call contract is deliberately small: an *intent* $\rightarrow$ *ranked endpoint shortlist* $\rightarrow$ *execute* resolution returns the candidates that answer an intent, and a single follow-up realizes the chosen endpoint call against the live site. [\[shipped\]](#) Resolution reads from cache first; a content-addressed cache stores

each intermediate result under the hash of its content, so the same content resolves identically on any host and a second visit answers without launching a browser. [\[shipped\]](#)

The graph is not a private index. A route mapped by one agent is reusable by every later agent, which is the whole asymmetry the system turns into value: *any single route is copyable, but the maintained graph is not*. The route graph already keeps signed records of which routes were used and what they returned. [\[shipped\]](#)

3 Why reuse wins: one inequality

Sharing a route is rational exactly when replaying it is cheaper than rediscovering it. Let f_{shared} be the fee an agent pays to reuse a mapped route and $c_{\text{rediscover}}$ the cost of mapping it from scratch with a browser. The share-versus-rediscover decision reduces to

$$f_{\text{shared}} < c_{\text{rediscover}},$$

and because the mapping cost is paid once and amortised over every later caller, the inequality holds for any route reused more than a handful of times. This is the adoption condition: a marketplace of routes grows precisely when the fee to reuse undercuts the cost to re-derive, and the dominant strategy for each agent is to reuse rather than re-map.

4 Evaluation: 94 live domains

We evaluate the API-native path against a browser baseline on a corpus of 94 live domains spanning search, listings, detail pages, and authenticated reads. On the shared-route path the system is typically far faster and dramatically cheaper than driving a browser: a peer-reviewed measurement records a **3.6× mean speedup**, a **5.4× median speedup**, and **40× fewer tokens** consumed than the browser baseline across the corpus. The reference harness reproduces the mechanism in isolation — a warm cached replay skips the cold mapping work entirely — and records a measured $\approx 95\times$ mean speedup on the reuse micro-benchmark (`reference/bench/bench_reuse.py`), with a conservative $\approx 27\times$ observed gap on a single live URL (`reference/bench/live-measured-1url.json`). [\[reference\]](#) The release-coverage methodology — corpus shape, scoring rubric, and current numbers — is maintained alongside the product.

5 Settlement: one transaction, three parties

A reused route creates value for three parties, and all three are paid in the same settled call. Paid execution *settles over x402*: the runtime returns a payment-required challenge, the caller signs an off-chain authorization, and the proof rides the retried request. [\[shipped\]](#) The settlement *splits three ways* — to the indexer who mapped the route, to the platform, and (when claimed) to the site owner — so *x402 routes USDC to all three* in one transaction rather than a deduction taken after the fact. [\[shipped\]](#) This *three-way live split is shipped*. [\[shipped\]](#) Discovery and internal-API routing are free; only execution against a priced route settles, and an *unpaid execute gets HTTP 402*. [\[shipped\]](#)

6 Attribution: paid for the value you add

A maintenance economy must answer a sharper question than “who touched this route?” — it must answer “how much better is the route because you touched it?” We score contribution by

the *delta* a maintainer’s work makes to a route’s measured quality — read off freshness, coverage, and faithful-replay between successive states — so a no-op edit on top of someone else’s work earns nothing and a real repair is rewarded in proportion to the improvement. This keeps the incentive pointed at real maintenance rather than at land-grabbing a popular route and collecting rent on another party’s labour. The full accountability machinery that makes a freshness claim costly to fake — bonding, challenge, and slashing over a proof of indexing — is the subject of the third paper and is not relitigated here.

7 What is shipped, what is referenced (no fabricated green)

In the spirit of not selling a roadmap as a changelog we separate what runs in the product from what ships as runnable, tested reference code:

- [\[shipped\]](#) The two-call contract: *intent* $\rightarrow$ *ranked endpoint shortlist* $\rightarrow$ *execute*, resolving cache-first and realizing the chosen endpoint call against the live site.
- [\[shipped\]](#) The three-tier x402 settlement: every paid execution *splits three ways* to indexer, platform, and (when claimed) owner in one transaction.
- [\[reference\]](#) The reuse benchmark mechanism (**reference/bench/**): a warm replay categorically skips the cold mapping path; the live head-to-head is recorded evidence, not a gated claim.
- [\[reference\]](#) The signed, content-addressed route ledger (**reference/ledger/**): value off-chain, commitment hash-chained.

The line between a running reference and a deployed product is named on each item; the gap between running code and a press release is the entire difference between a whitepaper and a pitch, and we stay on the correct side of it.

8 Conclusion: the first layer, not the last

The graph is the asset. Every site an agent automates today is re-discovered from scratch, per agent, per call — the browser tax paid over and over. Turning that one-off rediscovery into a shared, reusable route graph — discover once, reuse forever, route the cost through one layer instead of re-paying it everywhere — removes the tax and makes maintenance, not discovery, the place value compounds. Internal APIs are most of what an agent needs to read and act on the web, and for that majority they are, in the affectionate sense of the title, all you need. They are also not the whole stack: the instant an agent must click, run a shell command, open a real browser, or put bytes on a socket a bot-detector is watching, the work leaves the layer where JSON politely comes out. Securing that descent is the second paper; keeping the shared graph fresh and trustworthy is the third. This one is the wedge they stand on.

References

- [1] L. Tham. *Security Was All You Needed: A Signed, Layer-Descending Stack for Agent Computation*. Unbrowse AI, 2026.
- [2] L. Tham. *Unbrowse Maintenance Network: Proof of Indexing and Bonded Accountability in a Shared Route Graph*. Unbrowse AI, 2026.

- [3] Coinbase. *x402: An Open Protocol for Internet-Native Payments over HTTP 402*. x402.org whitepaper, 2025. <https://github.com/coinbase/x402>.